its constant value. Then there is no need to look up its value when it is referenced.

If a variable is defined as being "final" then it must be initialized in the same statement. It can never again be changed. By custom, it should be declared as all upper case. Any internal words in the identifier should be separated by underscores. If you try to change a variable that you earlier declared as being "final", then the compiler will complain. Obviously, it is better to have the compiler complain, than to have your program crash in production. Whether or not a variable is final is independent of its access. I.e., it can be declared either public or private.

```
private final int NUMBER_OF_MONTHS = 12;
final String FIRST_MONTH = "January";
```

4.5 Final methods and classes

When a method is declared to be final, Java knows that the method can never be overridden. A final method must be fully defined when it is declared. You cannot, for example, have an abstract final method. Since it is final, and can never be overridden by subclasses, the Java compiler can replace the call to the method with inline code if it wants. As for security, if you declare a method to be final, then you can be sure it isn't overridden. For example, class Object has a method called getClass() that is declared final. No subclass can override this method and thereby return some other class type to hide its identity.

A final class makes all of its methods final as well, since a final class cannot be extended. Examples of final classes are: Integer, Long, Float and Double. None of these "wrapper" classes can be subclassed. String is another class that's declared final. So, if you want to stop programmers from every making a subclass of a particular class, you can declare that class to be final.

4.6 Objects passed by reference

As we know, the name or reference for an object represents a memory location where the object is stored. When an object is passed, only the reference is passed. That means, only the address of the object is passed. A copy is *not* made of the object. This will have interesting implications.

Up until now, our classes have either been Applets or Applications. Now we create a class that is neither. This class cannot execute unless it is instantiated by either an Application or an Applet. First we create the class. Then we create another Applet/Application class to test it.